6-5 · Kafka Connect와 MinIO 적재 · 정리·회고 0.5H

주차 회고 · 산출물 점검

6주차를 되짚고, 7주차 RAG 로 넘어가는 다리

수도권ICT 이노베이션스퀘어

데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

30분 뒤에는

① 6주차 다섯 차시에서 남은 결과물을 한 장으로 말할 수 있다

② 이번 주에 또 나온 원칙 네 가지를 이전 주차와 잇는다

③ 자주 겪는 오류 다섯 가지의 원인과 해결을 안다

④ 컨테이너를 stop 으로 정돈하고 7주차 준비를 확인한다

한 주를 관통한 생각 — 보내는 쪽과 받는 쪽을 떼어 놓는다

오늘의 요약

6주차에 지나온 길 — 다섯 차시와 남은 결과물

차시	배운 것	남은 결과물
6-1	배치·스트리밍 · 토픽·파티션·오프셋	CLI 감각
6-2	리스너 2계층 · 컨슈머 그룹·LAG	docker-compose.yml
6-3	Producer/Consumer 코드 · 커밋	producer.py · consumer.py
6-4	CDC 와 Debezium	register-mysql.json
6-5	Sink 커넥터로 적재	sink-minio.json · cdc-raw

Kafka 가 사이에 들어가면 양쪽이 서로를 몰라도 되고, 받는 쪽이 늘거나 죽어도 보내는 쪽은 그대로

오늘의 요약

이번 주에 또 나온 원칙 네 가지

주소	컨테이너 안과 밖은 주소가 다르다 — 1-5·4-1·5-3 에 이어 6-2 리스너 설정
중복	중복은 막는 것이 아니라 견딘다 — 2-5 UNIQUE · 4-3 MERGE · 6-3 멱등한 수신
파티션	파티션은 병렬의 단위다 — 5주차 Spark 와 6주차 Kafka 가 같은 낱말을 쓴다
작은 파일	작은 파일이 쌓이면 나중에 느려진다 — 5-5 coalesce · 6-5 flush.size

같은 원칙이 주차를 바꿔 가며 다른 도구의 이름으로 되돌아온다

7주차로 이어지는 것

MinIO 의 raw 파일이 다음 주의 출발점

6주차	7주차 RAG	8주차 Airflow
MinIO 에 쌓은 raw 파일	문서 저장소와 같은 자리	파이프라인을 스케줄로 묶는다

RAG — 흘러온 데이터를 LLM 이 쓸 수 있게 만드는 일

실시간 CDC → 레이크하우스 적재는 8-3 흐름 자동화에서 다시 만난다

오늘의 체크리스트

여덟 개를 스스로 확인해 보세요

1. 소스/싱크 커넥터의 방향 차이를 설명했다
2. 싱크 커넥터가 사실상 코드 없는 컨슈머라는 것을 이해했다
3. store.url 이 왜 MinIO 를 가리키는지(S3 API 호환) 설명했다
4. connector-plugins 응답에서 S3SinkConnector 를 확인했다
5. cdc-raw 버킷을 만들고 커넥터를 RUNNING 으로 띄웠다
6. DB INSERT → MinIO JSON 파일 생성을 엔드투엔드로 확인했다
7. 파일이 안 생길 때 확인할 다섯 단계를 안다
8. 주차 산출물(Debezium·Sink JSON + compose + 토픽 구성)을 정리했다

자주 겪는 오류

증상으로 찾는 원인과 해결

증상	원인	해결
Failed to find any class … S3SinkConnector	플러그인이 경로에 없음 또는 재생성 안 함	마운트 경로 확인 → --force-recreate connect
Sink FAILED · trace 에 버킷 오류	cdc-raw 없음 · store.url 이 localhost	버킷 확인 · store.url 은 http://minio:9000
태스크 FAILED · Null valued records …	6-4 DELETE 뒤 툼스톤(값 없음)	behavior.on.null.values ignore → 다시 등록
RUNNING 인데 파일이 하나도 없다	flush.size 미달 · topics 오타	INSERT 더 · config 의 topics 를 목록과 대조
9000 충돌 · 이름 "/minio" 이미 사용	4·5주차 MinIO 살아 있음 · 4주차 minio 이름	그쪽 stop · 이름은 cdc-minio 로

포트를 9010 으로 바꿔도 store.url 은 그대로 http://minio:9000 — 커넥터는 컨테이너 이름으로 붙는다

예상 질문

자주 나오는 질문 네 가지

1. **Sink 대신 파이썬 Consumer 를 써도 되나요?**
   됩니다 — 다만 재시도·오프셋 관리·병렬화를 직접 짜야 해서 표준 적재는 커넥터가 안정적

2. **JSON 말고 Parquet 으로 못 쓰나요?**
   format.class 를 ParquetFormat 으로 — 스키마가 필요해서 설정이 조금 는다

3. **실시간인데 왜 파일로 쌓나요?**
   원본 보존과 재처리 — 실시간 소비는 Kafka 에서, 파일은 나중에 배치로 정제(람다 아키텍처)

4. **커넥터가 죽으면 데이터를 잃나요?**
   오프셋이 남아 다시 뜨면 이어서 읽는다 — 다만 flush 전 버퍼는 다시 읽어 중복될 수 있다

컨테이너 정돈

stop 만 — down 은 -v 없이도 토픽을 지운다

1. **cd ~/data-eng/week6 → docker compose stop**
   7주차는 메모리를 많이 쓴다 — 이번 주 것은 내려 둔다

2. **docker compose ps -a**
   넷이 Exited — stop · start 뒤에는 그대로(실측)

3. **mkdir -p ~/data-eng/week7**
   다음 주 폴더를 미리 만들어 둔다

down 하면

토픽·커넥터

그룹 삭제

> **7주차에 메모리가 부족하면 4·5·6주차 컨테이너를 모두 stop 해 두면 된다**

다음 회차 예고

7-1 · LangChain 챗봇 기초

01

이론 1H

LLM API 와

LangChain 구조

02

실습 1.5H

ChatModel 호출

프롬프트 템플릿

03

실습 1H

대화 이력을 가진

챗봇 실습

04

정리 0.5H

부품 정리 ·

7-2 문서 처리 준비

준비: LLM API 키(OpenAI 또는 Anthropic) 1개, 또는 로컬 Ollama + 모델 1개(ollama pull llama3.1) · Python 3.10 이상과 가상환경

6-5 · 정리·회고 0.5H 마무리

다음 회차는 7-1 LangChain 챗봇 기초

LLM 을 호출하고 대화 이력을 가진 콘솔 챗봇을 만듭니다 — 아직 우리 데이터는 쓰지 않습니다

- ✓ LLM API 호출 구조와 LangChain 의 역할
- ✓ ChatPromptTemplate 로 메시지 템플릿화
- ✓ prompt | llm | parser LCEL 체인
- ✓ 멀티턴 콘솔 챗봇 완성

준비할 것

API 키 또는 Ollama · Python 3.10+